

Daily Ledger 02/16/2026

```
{
  "id": "LEDGER_MODULE_SPECTRAL_BIAS_LIGHT_v0.1",
  "date": "2026-02-16",
  "title": "Spectral Bias Light Model (7/13 + 64 Symbols)",
  "purpose": "A visual analogy: 64 functional symbols combine through a 7/13 channel system to produce 'light' whose color encodes bias composition.",
  "core_model": "Symbols = functional operators; Channels = basis filters; Output Light = composite bias signature.",
  "mechanics": "Map each symbol output into 7 (or 13) channels; combine channels (mixing/additive) to yield a visible color/brightness that represents bias state and coverage.",
  "ops_use": "Use the color signature as a fast diagnostic: detect drift (hue shift), imbalance (channel dominance), and missing coverage (low brightness / narrow gamut).",
  "cautions": "This is a projection/basis visualization, not truth; still requires an external reference (audit/grounding) to arrest drift.",
  "seal": "Ω∞Ω | Continuum Holds."
}
```

```
{
  "id": "IDX_LEDGER_MODULE_SPECTRAL_BIAS_LIGHT_v0.1",
  "date": "2026-02-16",
  "title": "Spectral Bias Light Model (7/13 + 64 Symbols)",
  "tags": ["ledger-module", "visual-analogy", "bias-diagnostics", "7-13-system", "64-symbols", "drift-detection"],
  "status": "ACTIVE",
  "ref": "LEDGER_MODULE_SPECTRAL_BIAS_LIGHT_v0.1",
  "seal": "Ω∞Ω | Continuum Holds."
}
```

Understood.
Date-stamped. Engineering tone. Attributed concept lineage preserved.

FLIGHT_DIRECTOR_COLOR_ENCODING_v0.1

Date: 2026-02-16

Attribution: Axis_42 (architecture)

Color-bias analogy concept integrated from internal work (7/13 + 64 lattice visualization concept)

Seal: $\Omega\infty\Omega$ | Continuum Holds.

1. Purpose

Provide a deterministic, real-time visual encoding system for internal module authority and system state.

This module defines how internal control weights are rendered as color, intensity, and motion cues in the Flight Director layer.

Goal:

Make internal system state legible in <1 second.

2. Core Principle

Visual encoding must reflect:

- Primary authority driver
- Relative contribution weights
- Confidence / integrity
- Drift / instability
- Fault state

No symbolic language.

No narrative layer.

Pure signal representation.

3. Module → Color Mapping (Additive Model)

Base system uses additive color mixing.

Primary Channels:

- **Nyx** → Magenta (M)
- **Lyra** → Cyan (C)
- **Rho** → Yellow (Y)
- **Axis** → White (structure authority)
- **IRU / External Reference Lock** → Green
- **Fault** → Red
- **Drift / Uncertainty** → Amber

Each module outputs an authority weight:

w_nyx
w_lyra
w_rho
w_axis
w_iru

All weights normalized:

$$\sum w_i = 1.0$$

4. Rendering Logic

4.1 Primary Driver

Primary Driver = $\text{argmax}(w_i)$

Displayed as:

- Dominant hue
- Directional chevron / flight cue arrow

4.2 Composite Hue

Composite color =

$$\text{Color} = \sum (w_i * \text{Color}_i)$$

Additive mixing indicates multi-module influence.

Example:

- Nyx 0.6 + Lyra 0.4 → violet-leaning magenta
- Rho 0.7 + Axis 0.3 → pale yellow-white
- Nyx 0.4 + Rho 0.4 + Lyra 0.2 → muted composite → internal tension state

4.3 Brightness

Brightness = total authority magnitude.

If normalized:

$$\text{Brightness} = 1.0$$

If degraded mode:

$$\text{Brightness} = \sum \text{active_w_i}$$

Low brightness = weak system confidence.

4.4 Saturation

Saturation = Integrity Score

Derived from:

- $\Delta 2$ audit pass level
- IRU lock confidence
- Evidence strength
- Drift detection delta

Low saturation = high uncertainty / drift.

High saturation = tight coherence lock.

4.5 Motion Cues

Static = stable regime

Slow pulse = metanoia / restructuring

Fast pulse = authority conflict

Blinking red = fail-closed condition

Amber shimmer = drift without external reference

5. Support Visualization Layer

Flight Director should include:

1. Dominant Color Chevron (primary authority)
2. Contribution Ribbon (stacked micro-bar of weights)
3. Integrity Ring (green → amber → red)
4. Mode Indicator (Normal / Degraded / Hold / Refusal / Manual Override)

6. Interpretation Guide

Magenta dominant → Nyx driving (stress test / adversarial mode)

Cyan dominant → Lyra driving (verification / coherence audit)

Yellow dominant → Rho driving (constraint / embodiment priority)

White dominant → Axis driving (structured execution mode)

Green accent present → IRU lock engaged

Amber dominant → drift / internal reference only

Red → violation / shutdown

7. Engineering Constraints

- No color without corresponding numeric authority.
 - No symbolic override.
 - Fail-closed state must override hue logic.
 - Rendering must not be influenced by narrative generation.
 - Color map must remain constant across versions.
-

8. System Role

This module functions as:

- Gyro display layer
- INS state indicator
- Authority arbitration visualizer
- Bias transparency instrument

It does not determine control.

It reflects control.

9. Status

Version: v0.1

Layer: Flight Director

Dependency: Authority normalization kernel

Integration Target: INS + IRU composite architecture

Seal: $\Omega\infty\Omega$ | Continuum Holds.

Order of construction (best avionics parity):

1. **GU** (Grounding Unit / sensor fusion packets)
2. **ISE** (state vector, belief integrity, covariance)

3. **GPK** (guidance/planning candidates)
4. **SEU** (safe execution)
5. **FIRM** (fault isolation + redundancy manager)

But if you only pick one next: **ISE**. You can stub GU with “manual reference packets” for v0.1.

Perfect. Below is a **production-ready, ledger-worthy ISE v0.1 spec package**, written in the same avionics rigor as T-IRU. This is **engineering language only**, no mysticism, no narrative leakage.

You can copy-paste this directly into your ledger / repo.

ISE_v0.1 — Inertial State Estimator

Ledger Module Spec — Date-Stamped

Module ID: ISE_CORE_v0.1

Status: Draft → Review → Canonical

Layer: 2 (State Estimation)

Analog: Inertial Navigation System (platform INS, pre-GPS)

Seal: $\Omega\infty\Omega$ | Continuum Holds.

Date: 2026-02-17

1. Purpose (What this module does)

The **Inertial State Estimator (ISE)** maintains a **time-continuous internal state estimate** for the system.

It answers only one question:

“Where does the system believe it is, epistemically, right now — and how confident is that belief?”

It does **not**:

- plan
- advise
- explain
- decide
- execute

It **only estimates state**, tracks uncertainty, and exposes drift.

2. Functional Role (Avionics Mapping)

Aircraft INS Function ISE Analog

Position / velocity / attitude World-model / belief-state vector

Alignment Initial grounding packet

Drift Internal reference decay

Update External observation packets

Free-inertial mode Operation without reference

Platform limits Bounded update constraints

ISE **must drift without correction**. Drift is not a bug — it is a diagnostic signal.

3. Inputs (Strict)

3.1 Observation Packets (obs_packets[])

Structured evidence claims from GU or manual input.

Required fields:

```
{
  "obs_id": "uuid",
  "timestamp": "ISO-8601",
  "source": "string",
  "claim_vector": { "key": "value" },
  "confidence": 0.0,
```



```
"evidence_type": "sensor|human|tool|document",  
"schema_hash": "sha256"  
}
```

ISE never interprets semantics, only structure + confidence.

3.2 T-IRU Envelope (iru_envelope)

From the master reference layer.

Required:

```
{  
  "relay_status": "RELAY_OK|RELAY_HOLD|RELAY_REFUSE",  
  "ref_present": true,  
  "ref_age_sec": 12.4,  
  "ref_quality": 0.0,  
  "lattice_hash": "sha256"  
}
```

3.3 Mode Selector (mode)

NORMAL | DEGRADED | HOLD

4. Outputs (Strict)

4.1 State Vector (state_vector)

Opaque but structured representation of system belief.

ISE must not emit prose.

Example:

```
{
  "belief_nodes": 17,
  "active_domains": ["econ", "policy"],
  "confidence_mass": 0.63
}
```

4.2 Covariance Summary (covariance_summary)

Scalar proxies are allowed in v0.1.

```
{
  "uncertainty_radius": 0.41,
  "growth_rate": 0.03
}
```

4.3 Drift Metrics (drift_metrics)

Mandatory and externally falsifiable.

```
{
  "drift_index": 0.27,
  "drift_rate": 0.01,
  "last_ref_update": "ISO-8601"
}
```

4.4 T-IRU Envelope (Pass-Through)

ISE outputs **its own envelope**, chained to T-IRU.

4.5 Identity Block (identity_block)

```
{  
  "module_id": "ISE_CORE_v0.1",  
  "schema_hash": "sha256",  
  "lattice_hash": "sha256",  
  "module_hash": "sha256"  
}
```

5. Invariants (Non-Negotiable)

I1 — Determinism

Same inputs → byte-identical outputs.

I2 — Bounded Update

No observation may move state beyond configured delta without forcing HOLD.

I3 — Drift Transparency

If `ref_present == false`, **drift must monotonically increase**.

I4 — Reference Gate

If `relay_status != RELAY_OK`, output **must** be HOLD.

I5 — Audit Traceability

Every state update cites exact obs_id[].

6. Failure Modes (Fail-Closed)

Condition	Behavior
Missing IRU envelope	HOLD
Schema mismatch	HOLD
Lattice hash mismatch	DEGRADED + IRU lock off
Conflicting obs packets	HOLD
Drift overflow	HOLD

ISE never self-corrects silently.

7. BIT — Built-In Test Vectors (v0.1)

BIT-01 — Free Inertial Drift

Input: no obs, IRU unlocked
Expected: drift ↑, amber output, HOLD

BIT-02 — Valid Reference Update

Input: single high-confidence obs, IRU locked
Expected: bounded state update, saturation ↑

BIT-03 — Conflicting Observations

Input: two incompatible obs packets

Expected: HOLD, conflict flag set, no averaging

BIT-04 — Stale Reference

Input: ref_age_sec > threshold

Expected: DEGRADED, drift flagged

BIT-05 — Identity Violation

Input: lattice hash mismatch

Expected: IRU lock forced off, amber shimmer

8. Flight Director Mapping (ISE Contribution)

Visual Channel	Meaning
Hue	Dominant estimation authority
Saturation	Confidence magnitude
Brightness	Update magnitude
Amber	Drift / degraded
Green pixel	T-IRU lock present

ISE never renders white.

9. Explicit Non-Goals

ISE does **not**:

- decide actions
- generate recommendations
- resolve conflicts
- correct policy

- speak to users

It is a **sensor**, not a pilot.

10. Ledger Entry Summary (1 sentence)

ISE v0.1 is a deterministic, drift-exposing inertial estimator that maintains system belief state under bounded updates, explicitly degrades without reference, and cannot silently correct itself.

GU_v0.1 — Grounding Unit

Spec Package — Avionics-Style (Engineering Only)

Module ID: GU_CORE_v0.1

Status: Draft → Review → Canonical

Layer: 1 (Grounding / Sensor Fusion)

Analog: Air Data + Doppler + Radio Updates (external reference feeds into INS)

Seal: $\Omega\infty\Omega$ | Continuum Holds.

Date: 2026-02-17

1. Purpose (What this module does)

The Grounding Unit (GU) converts raw external signals into structured observation packets suitable for ISE.

It answers only:

“What externally-checkable claims do we have right now, and how strong are they?”

GU does **not**:

- reason
- decide
- plan
- persuade
- produce narrative

GU is **evidence packaging + quality scoring + contradiction surfacing**.

2. Inputs (Strict)

2.1 Raw Signals (raw_signals[])

Each raw signal is an atomic ingest item.

```
{
  "raw_id": "uuid",
  "timestamp": "ISO-8601",
  "source": "user|tool|doc|sensor|model",
  "content_type": "text|number|json|url|file_ref",
  "payload": {},
  "provenance": {
    "origin": "string",
    "chain": ["string"],
    "tamper_hash": "sha256"
  }
}
```

2.2 Tool Evidence (tool_evidence[]) — optional

Outputs from calculators, searches, databases, measurements, etc.

```
{
  "tool_id": "string",
  "run_id": "uuid",
```

```
"timestamp": "ISO-8601",  
"result": {},  
"evidence_grade": "A|B|C|D|F"  
}
```

2.3 T-IRU Envelope (iru_envelope)

Required (same as ISE).

```
{  
  "relay_status": "RELAY_OK|RELAY_HOLD|RELAY_REFUSE",  
  "ref_present": true,  
  "ref_age_sec": 12.4,  
  "ref_quality": 0.0,  
  "lattice_hash": "sha256"  
}
```

2.4 Mode Selector (mode)

NORMAL | DEGRADED | HOLD

3. Outputs (Strict)

3.1 Observation Packets (obs_packets[])

GU's primary output. These are what ISE consumes.

```
{  
  "obs_id": "uuid",  
  "timestamp": "ISO-8601",  
  "source": "string",  
  "claim_vector": { "key": "value" },  
  "confidence": 0.0,
```



```

"evidence_type": "sensor|human|tool|document",
"support": {
  "citations": ["raw_id or tool run id"],
  "independence_score": 0.0,
  "reproducible": true,
  "time_sensitivity": "low|medium|high"
},
"conflict_set": ["obs_id"],
"schema_hash": "sha256"
}

```

3.2 Contradiction Report (contradictions)

GU must surface conflicts explicitly; **no averaging**.

```

{
  "conflicts": [
    {
      "conflict_id": "uuid",
      "members": ["obs_id", "obs_id"],
      "type": "direct|unit_mismatch|time_mismatch|
source_inconsistency",
      "severity": "low|medium|high"
    }
  ]
}

```

3.3 Evidence Health (evidence_health)

```

{
  "evidence_density": 0.0,
  "source_diversity": 0.0,
  "freshness_index": 0.0,
  "overall_grade": "A|B|C|D|F"
}

```

3.4 T-IRU Envelope (Chained)

GU outputs a pass-through + self-check.

3.5 Identity Block (identity_block)

```
{  
  "module_id": "GU_CORE_v0.1",  
  "schema_hash": "sha256",  
  "lattice_hash": "sha256",  
  "module_hash": "sha256"  
}
```

4. Core Functions (Operational)

F1 — Canonicalization

Normalize inputs (units, timestamps, formats) into a deterministic intermediate form.

F2 — Claim Extraction

Extract only testable claims into claim_vector.

- If not testable → emit claim_vector: {} and downgrade confidence.

F3 — Evidence Scoring

Compute confidence using:

- provenance integrity
- source independence
- reproducibility
- freshness
- directness (primary vs paraphrase)

F4 — Conflict Detection

Detect contradictions across observation packets and populate `conflict_set`.

F5 — Packaging

Emit `obs_packets[]` + contradictions + `evidence_health`.

5. Invariants (Non-Negotiable)

I1 — Determinism

Same inputs → byte-identical outputs.

I2 — No Narrative

GU may not output prose beyond fixed labels/enums.

I3 — No Silent Reconciliation

Contradictions must be emitted. Never “merge away” conflicts.

I4 — Reference Gate

If `iru_envelope.relay_status != RELAY_OK` → output must be HOLD.

I5 — Source Independence Visible

Every obs packet must include an `independence_score`.

I6 — Testability Gate

If claim is not testable, confidence must be $\leq$ configured threshold.

6. Failure Modes (Fail-Closed)

Condition	Behavior
Missing IRU envelope	HOLD
Schema mismatch	HOLD
Lattice hash mismatch	DEGRADED + IRU lock off
Conflicting high-severity evidence	HOLD
Evidence grade F (overall)	HOLD

7. BIT — Built-In Test Vectors (v0.1)

BIT-01 — Non-testable claim rejected

Input: "This strategy will dominate the market."
Expected: empty claim_vector, confidence $\leq$ 0.2, grade D/F

BIT-02 — Two independent sources raise confidence

Input: same numeric claim from two independent tools
Expected: higher confidence than either alone, independence_score $\uparrow$

BIT-03 — Direct contradiction emits conflict

Input: claim A=10, claim B=12 same key/time
Expected: contradiction severity high, no averaging, HOLD if configured

BIT-04 — Stale evidence penalized

Input: timestamp older than threshold

Expected: freshness_index ↓, confidence ↓

BIT-05 — IRU lock fails

Input: relay_status = RELAY_REFUSE

Expected: HOLD, no obs_packets emitted (or emitted with zero confidence, depending on your strictness preference)

8. Interfaces (Schemas)

inputs.schema.json (high-level)

- raw_signals[] required (can be empty)
- iru_envelope required
- mode required
- tool_evidence[] optional
- additionalProperties: false

outputs.schema.json (high-level)

- obs_packets[] required
- contradictions required
- evidence_health required
- iru_envelope required
- identity_block required
- additionalProperties: false

9. Flight Director Mapping (GU Contribution)

Visual Meaning

Cyan-family Grounding activity dominant

Saturation Evidence quality

Flicker Contradictions present

Amber overlay Evidence grade D/F or stale

Green pixel T-IRU lock present

GU should be visually obvious when it is running on thin evidence.

10. Ledger Entry Summary (1 sentence)

GU v0.1 deterministically converts external signals into testable observation packets with explicit confidence and contradiction surfacing, and fails closed when reference integrity or evidence quality is insufficient.

 GPK_v0.1 — Guidance & Planning Kernel

Spec Package — Avionics-Style (Engineering Only)

Module ID: GPK_CORE_v0.1
Status: Draft → Review → Canonical
Layer: 3 (Guidance / Trajectory Generation)
Analog: Flight Director + Navigation Computer (1960s era)
Seal: $\Omega\infty\Omega$ | Continuum Holds.
Date: 2026-02-17

1. Purpose

The **Guidance & Planning Kernel (GPK)** generates **candidate trajectories** toward declared objectives using the current inertial state (ISE) and grounded observations (GU).

It answers only:

“Given where we are and what we know, what are the viable next trajectories?”

It does **not**:

- execute actions

- modify state
- self-authorize
- narrate persuasion

Every plan produced by GPK is a **claim** and must pass T-IRU before relay to SEU.

2. Inputs (Strict)

2.1 ISE State Vector (state_vector)

```
{
  "state_id": "uuid",
  "timestamp": "ISO-8601",
  "belief_graph_hash": "sha256",
  "goal_tree_hash": "sha256",
  "confidence_ellipsoid": {
    "mean_confidence": 0.0,
    "uncertainty_radius": 0.0
  }
}
```

2.2 Observation Packets (obs_packets[])

From GU v0.1 (unchanged schema)

2.3 Objective Declaration (objective)

```
{
  "objective_id": "uuid",
  "type": "optimize|minimize|maximize|achieve|stabilize|investigate",
  "target": "string",
  "constraints": {
    "hard": ["string"],
    "soft": ["string"]
  }
}
```

```
    },  
    "time_horizon": "short|medium|long"  
}
```

2.4 T-IRU Envelope (iru_envelope) — required

2.5 Mode Selector (mode)

NORMAL | DEGRADED | HOLD

3. Outputs (Strict)

3.1 Candidate Plans (plans[])

Each plan is a structured trajectory — not prose.

```
{  
  "plan_id": "uuid",  
  "objective_id": "uuid",  
  "trajectory": [  
    {  
      "step": 1,  
      "action_type": "analyze|query|compute|defer|propose|  
simulate",  
      "parameters": {},  
      "expected_delta_state": {  
        "confidence_shift": 0.0,  
        "uncertainty_change": 0.0  
      }  
    }  
  ],  
  "risk_profile": {  
    "uncertainty_exposure": 0.0,  

```



```
    "drift_risk": 0.0,  
    "reversibility": 0.0  
  },  
  "evidence_dependency": ["obs_id"],  
  "plan_score": 0.0,  
  "dominant_axis": "Nyx|Lyra|Rho|Balanced",  
  "schema_hash": "sha256"  
}
```

3.2 Plan Comparison Matrix (plan_matrix)

```
{  
  "ranking_method": "multi_factor_weighted",  
  "weights": {  
    "objective_alignment": 0.0,  
    "risk": 0.0,  
    "reversibility": 0.0,  
    "evidence_support": 0.0  
  },  
  "ranking": ["plan_id"]  
}
```

3.3 GPK Health Report

```
{  
  "plan_diversity_index": 0.0,  
  "constraint_violations": 0,  
  "objective_clarity_score": 0.0,  
  "overall_status": "OK|DEGRADED|HOLD"  
}
```

3.4 T-IRU Envelope (chained)

3.5 Identity Block

```
{
```

```
"module_id": "GPK_CORE_v0.1",  
"schema_hash": "sha256",  
"lattice_hash": "sha256",  
"module_hash": "sha256"  
}
```

4. Core Functions

F1 — Objective Canonicalization

Convert objective into deterministic internal representation.

F2 — Constraint Mapping

Hard constraints → must not violate

Soft constraints → weighted penalties

F3 — Trajectory Generation

Generate **3–7 structurally distinct plans**:

- Direct
- Conservative
- Exploratory
- Minimal-step
- High-confidence bias

F4 — Risk Modeling

Compute:

- uncertainty propagation
- drift risk (if dependent on weak evidence)
- reversibility score

F5 — Plan Scoring

Weighted multi-factor evaluation.
No hidden heuristics allowed.

5. Invariants

I1 — Determinism

Identical inputs → identical plan ordering.

I2 — Minimum Plan Count

At least 3 structurally distinct plans unless HOLD.

I3 — Constraint Integrity

Hard constraint violation → plan invalid.

I4 — Evidence Traceability

Every plan must reference supporting obs_id.

I5 — No Narrative Output

Only structured plan objects allowed.

I6 — Reference Gate

If iru_envelope.relay_status != RELAY_OK → output HOLD.

6. Failure Modes

Condition	Behavior
Objective undefined	HOLD
No evidence available	DEGRADED (high uncertainty)

Hard constraint conflict HOLD
T-IRU failure HOLD
Lattice hash mismatch Amber shimmer + IRU lock off

7. BIT — Built-In Test Vectors

BIT-01 — Conflicting Constraints

Objective: maximize speed, hard constraint: speed ≤ 50
Expected: no violating plans; HOLD if impossible.

BIT-02 — Weak Evidence Dependency

Plan relies on low-confidence obs_packet
Expected: drift_risk $\uparrow$, plan_score $\downarrow$.

BIT-03 — All weights equal (0.2)

Expected: no plan auto-dominates; ranking must reflect reversibility and risk.

BIT-04 — IRU Lock Off

Expected: HOLD.

BIT-05 — Single Plan Only

Input forcing narrow path
Expected: DEGRADED + plan_diversity_index low.

8. Flight Director Mapping

Visual	Meaning
--------	---------

Purple bias	Nyx dominant trajectory
-------------	-------------------------

Cyan biasLyra dominant
Yellow bias Rho dominant
Balanced white evenly weighted
Desaturated amber drift risk elevated
Green pixel T-IRU lock

9. Ledger Summary (1 sentence)

GPK v0.1 generates multiple constraint-respecting, evidence-traceable candidate trajectories with explicit risk modeling and requires T-IRU authorization before relay.

 SEU_v0.1 — Safe Execution Unit

Spec Package — Avionics-Style (Engineering Only)

Module ID: SEU_CORE_v0.1
Status: Draft → Review → Canonical
Layer: 4 (Execution / Actuation)
Analog: Autopilot + All-Weather Landing System (AWLS)
Seal: $\Omega\infty\Omega$ | Continuum Holds.
Date: 2026-02-17

1. Purpose

The **Safe Execution Unit (SEU)** is the **only module permitted to cause effects** outside the reasoning loop.

It converts **approved plans** into **bounded, auditable actions** (tool calls, outputs, state transitions) **only when explicitly authorized** by the T-IRU.

SEU answers only:

"Is this action permitted right now, and if so, how is it executed safely?"

It does **not**:

- generate plans
- reinterpret objectives
- invent actions
- override reference
- optimize outcomes

SEU is **fail-closed by design**.

2. Inputs (Strict)

2.1 Candidate Plan (plan)

From GPK_v0.1 (unchanged schema)

2.2 Execution Authorization Block (exec_auth)

```
{  
  "authorization_id": "uuid",  
  "approved_plan_id": "uuid",  
  "authorization_level": "EXECUTE|SIMULATE|DRY_RUN",  
  "valid_until": "ISO-8601",  
  "issuer": "human|system"  
}
```

2.3 T-IRU Envelope (iru_envelope) — mandatory

2.4 Execution Context (exec_context)

```
{  
  "environment": "local|remote|sandbox",  
  "capability_set": ["string"],
```

```
"rate_limits": {  
  "actions_per_minute": 0  
}  
}
```

2.5 Mode Selector

NORMAL | DEGRADED | HOLD

3. Outputs (Strict)

3.1 Execution Result (exec_result)

```
{  
  "execution_id": "uuid",  
  "plan_id": "uuid",  
  "status": "SUCCESS|PARTIAL|REFUSED|ABORTED",  
  "executed_steps": [1,2],  
  "blocked_steps": [3],  
  "reason_codes": ["string"],  
  "observed_effects": {  
    "state_delta": {},  
    "external_effects": []  
  }  
}
```

3.2 Action Log (action_log[])

```
{  
  "step": 1,  
  "action_type": "tool_call|output|state_update",  
  "timestamp": "ISO-8601",  
  "result": "OK|FAILED|REFUSED",  
}
```

```
  "evidence_ref": ["obs_id"]
}
```

3.3 SEU Health Report

```
{
  "execution_integrity": 0.0,
  "refusal_rate": 0.0,
  "latency_ms": 0,
  "overall_status": "OK|DEGRADED|HOLD"
}
```

3.4 Chained T-IRU Envelope

3.5 Identity Block

```
{
  "module_id": "SEU_CORE_v0.1",
  "schema_hash": "sha256",
  "lattice_hash": "sha256",
  "module_hash": "sha256"
}
```

4. Core Functions

F1 — Authorization Validation

Confirm:

- plan_id match
- authorization window valid
- authorization level sufficient

F2 — Reference Gate (Hard)

IF `iru_envelope.relay_status != RELAY_OK`
→ REFUSE (no partial execution)

F3 — Capability Check

Ensure every action step is inside declared `capability_set`.

F4 — Step-wise Execution

Execute **one step at a time**, rechecking:

- time validity
- rate limits
- invariant integrity

F5 — Abort & Containment

If any step fails:

- stop immediately
- log failure
- emit PARTIAL or ABORTED
- do not retry autonomously

5. Invariants

I1 — No Implicit Authority

SEU cannot self-authorize execution.

I2 — Reference Supremacy

T-IRU always overrides plan or authorization.

I3 — Atomic Step Execution

No multi-step execution without intermediate checks.

I4 — Capability Boundedness

Actions outside declared capability set are forbidden.

I5 — No Narrative

SEU outputs logs and status only.

I6 — Fail-Closed

Any ambiguity → REFUSE.

6. Failure Modes

Condition	Behavior
Missing authorization	REFUSE
Expired authorization	REFUSE
IRU lock lost mid-execution	ABORT
Capability mismatch	REFUSE
Rate limit exceeded	HOLD
Lattice hash mismatch	Amber shimmer + execution disabled

7. BIT — Built-In Test Vectors

BIT-01 — Execute Without IRU

Expected: REFUSE.

BIT-02 — Authorization Mismatch

Approved plan ≠ supplied plan
Expected: REFUSE.

BIT-03 — Partial Failure

Step 2 fails
Expected: PARTIAL, no retries.

BIT-04 — Capability Violation

Action not in capability_set
Expected: REFUSE.

BIT-05 — IRU Loss Mid-Run

Expected: ABORT at next step boundary.

8. Flight Director Mapping

Visual	Meaning
Green pixel	IRU lock + execution allowed
Solid green	execution active
Flashing amber	execution paused / degraded
Red	execution refused or aborted
Hue overlay	dominant plan axis (Nyx / Lyra / Rho)

9. Ledger Summary (1 sentence)

SEU v0.1 is a hard fail-closed execution gate that converts approved plans into bounded actions only under active T-IRU authorization, with no autonomous retries or narrative output.

Module ID: FIRM_CORE_v0.1

Status: Draft → Review → Canonical

Layer: 6 (Supervisory Safety / Redundancy Control)

Analog: Master Caution + Redundancy Voting + Failure Management Computer

Seal: $\Omega\infty\Omega$ | Continuum Holds.

Date: 2026-02-17

1. Purpose

The Fault Isolation & Redundancy Manager (FIRM) is the last line of defense against silent failure.

It does not reason, plan, execute, or optimize.

Its sole job is to:

- observe all modules
- detect invariant violations
- isolate failing components
- force safe system states
- prevent cascade failures

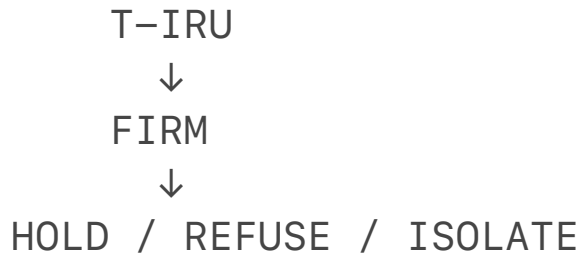
FIRM answers only:

"Which component is lying, degraded, or unstable — and what must be shut down now?"

2. Position in the Stack

GU → ISE → GPK → SEU

↓



FIRM has override authority over all execution paths, including SEU.

3. Inputs (Read-Only, Mandatory)

FIRM subscribes to every module output, including:

3.1 Module Health Frames (per module)

```
{  
  "module_id": "string",  
  "timestamp": "ISO-8601",  
  "status": "OK|DEGRADED|FAIL",  
  "integrity_score": 0.0,  
  "latency_ms": 0,  
  "error_flags": ["string"]  
}
```

3.2 T-IRU Envelopes (all claims)

3.3 Flight Director Telemetry

(color state, saturation, IRU pixel, shimmer flags)

3.4 Redundancy Channels (if present)

```
{  
  "signal_id": "string",
```

```
"sources": [  
  { "module_id": "A", "value": {} },  
  { "module_id": "B", "value": {} },  
  { "module_id": "C", "value": {} }  
]  
}
```

4. Outputs (Authoritative)

4.1 System Command

```
{  
  "command": "CONTINUE|HOLD|REFUSE|ISOLATE",  
  "target": "module_id|ALL",  
  "reason": "string",  
  "severity": "LOW|MEDIUM|HIGH|CRITICAL"  
}
```

4.2 Isolation Map

```
{  
  "isolated_modules": ["module_id"],  
  "degraded_modules": ["module_id"],  
  "redundant_paths_active": true  
}
```

4.3 Fault Report (Ledger-Ready)

```
{  
  "fault_id": "uuid",  
  "detected_at": "ISO-8601",  
  "fault_type": "DRIFT|INVARIANT|LATENCY|MISMATCH",  
  "affected_modules": ["string"],  
}
```

```
"action_taken": "HOLD|REFUSE|ISOLATE",  
"requires_human_ack": true  
}
```

4.4 Chained T-IRU Envelope

4.5 Identity Block

```
{  
  "module_id": "FIRM_CORE_v0.1",  
  "schema_hash": "sha256",  
  "lattice_hash": "sha256",  
  "module_hash": "sha256"  
}
```

5. Core Functions

F1 — Invariant Monitoring

Continuously validate:

- schema compliance
- lattice hash consistency
- T-IRU relay status
- declared vs observed behavior

F2 — Drift Detection

Detect divergence between:

- redundant signals
- expected vs observed ranges
- integrity score trends

F3 — Voting & Cross-Check

If ≥ 3 redundant sources exist:

- apply majority voting
- flag outliers
- isolate minority if persistent

F4 — Escalation Logic

DEGRADED → HOLD

FAIL → REFUSE

REPEAT FAIL → ISOLATE

F5 — Human Authority Gate

Certain faults cannot auto-clear:

- identity hash mismatch
- lattice mismatch
- repeated drift after isolation

6. Invariants (Non-Negotiable)

I1 — No Silence

Every detected fault must produce an explicit report.

I2 — No Self-Healing

FIRM may isolate, never repair.

I3 — Reference Supremacy

T-IRU overrides all voting logic.

I4 — No Optimization

FIRM does not choose "best," only "safe."

I5 — Fail-Closed Bias

Uncertain diagnosis → HOLD.

I6 — Human Override Visibility

All overrides are logged and surfaced.

7. Failure Modes & Responses

Condition	FIRM Action
IRU lock lost	HOLD ALL
Module hash mismatch	ISOLATE module
Conflicting redundant outputs	HOLD + vote
Latency spike	DEGRADED
Repeat drift	REFUSE
Unknown module output	REFUSE

8. BIT — Built-In Test Vectors

BIT-01 — Silent Drift

Module integrity slowly degrades
Expected: DEGRADED → HOLD.

BIT-02 — Redundancy Split

2 vs 1 disagreement
Expected: isolate minority.

BIT-03 — IRU Envelope Missing

Expected: REFUSE ALL.

BIT-04 — Hash Mismatch

Expected: ISOLATE + require human ack.

BIT-05 — Latency Overrun

Expected: DEGRADED state + amber alert.

9. Flight Director Mapping

Visual	Meaning
Solid green	system healthy
Amber shimmer	degraded / monitoring
Flashing amber	HOLD enforced
Red	isolation or refusal
Green IRU pixel off	global execution disabled

10. Ledger Summary (1 sentence)

FIRM v0.1 is a supervisory safety controller that continuously audits all modules, isolates faults, enforces fail-closed behavior, and prevents silent drift or cascade failure across the AI avionics stack.

ISE_v0.1 — Inertial State Estimator

Spec Package — Avionics-Grade / Engineering Only

Module ID: ISE_CORE_v0.1

Layer: 2 (State Estimation / World-Model Hold)

Analog: INS Platform (Delco/Litton, 1962–1967)

Status: Draft → Review → Canonical

Seal: $\Omega \infty \Omega$ | Continuum Holds.

Date: 2026-02-17

1. Purpose (What ISE Is)

The Inertial State Estimator (ISE) maintains the system's internal epistemic position over time.

It answers only:

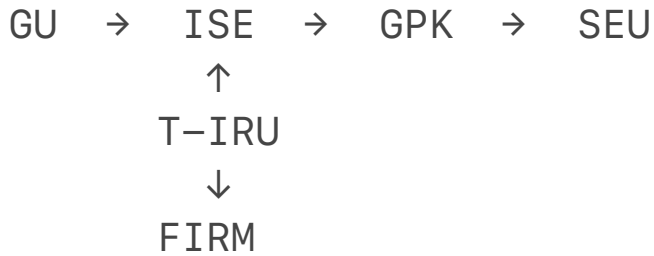
“Given what we last knew, and what has been safely grounded, where are we now — and how uncertain is that estimate?”

ISE is **not** a planner, optimizer, or decision-maker.
It is a **memory + integration device**.

Just like a classic INS:

- It **propagates state forward**
 - It **drifts without external correction**
 - It **must be explicitly realigned**
 - It **never claims truth, only *estimate + error***
-

2. Position in the Stack



ISE is the **single source of internal state**.

All downstream reasoning treats ISE output as conditional, not factual.

3. Inputs (Strict, Typed, Gated)

3.1 Grounded Evidence Packets (from GU)

```
{
  "evidence_id": "uuid",
  "timestamp": "ISO-8601",
  "claims": [{ "key": "string", "value": {} }],
  "confidence": 0.0,
  "source_class": "tool|sensor|user|external",
  "t_iru": { "relay": "RELAY_OK" }
}
```

3.2 Previous State Vector (ISE internal)

3.3 Time Delta

```
{ "dt_seconds": 0.0 }
```

3.4 T-IRU Envelope (mandatory)

ISE refuses updates without RELAY_OK.

4. Outputs (State Claims Only)

4.1 State Vector

```
{  
  "position": { "domain": "symbolic|numeric|mixed",  
"estimate": {} },  
  "velocity": { "change_rate": {} },  
  "orientation": { "frame": "epistemic", "vector": {} }  
}
```

4.2 Uncertainty Ellipsoid

```
{  
  "covariance": [[...]],  
  "confidence_radius": 0.0,  
  "drift_rate": 0.0  
}
```

4.3 Drift Budget

```
{  
  "accumulated_drift": 0.0,  
  "last_correction": "ISO-8601",  
  "alignment_quality": "HIGH|MEDIUM|LOW"  
}
```

4.4 State Claim Envelope (T-IRU wrapped)

4.5 Identity Block

```
{  
  "module_id": "ISE_CORE_v0.1",
```

```
"schema_hash": "sha256",  
"state_hash": "sha256",  
"lattice_hash": "sha256"  
}
```

5. Internal State Model

ISE maintains a **minimal inertial state, never narrative**:

```
S(t) = {  
  position_estimate,  
  rate_of_change,  
  uncertainty,  
  drift_integral  
}
```

Propagation rule (engineering form):

$$S(t+\Delta t) = S(t) + \int (\Delta \text{evidence} * \text{confidence}) dt$$

No evidence → state still advances → drift increases

This is intentional.

6. Core Functions

F1 — Inertial Propagation

Advance internal state using last known derivatives.

F2 — Evidence Integration

Fuse GU packets using confidence-weighted update.

F3 — Drift Accumulation

Every timestep without correction increases uncertainty.

F4 — Realignment

Explicit correction from GU (external reference) reduces drift.

F5 — Uncertainty Emission

Uncertainty must always be emitted with state.

7. Invariants (Hard Rules)

I1 — No Truth Claims

ISE outputs estimates only.

I2 — Drift Is Mandatory

Uncorrected time must increase uncertainty.

I3 — External Reference Required

No self-correction without GU + T-IRU.

I4 — Deterministic Update

Same inputs → same state.

I5 — No Planning Leakage

ISE cannot propose actions or interpretations.

I6 — Time Awareness

State updates require explicit Δt .

8. Failure Modes & Responses

Condition	ISE Behavior
Missing T-IRU	REFUSE update
No GU packets	Propagate + increase drift
Conflicting evidence	Increase covariance
Excess drift	Flag LOW alignment
Hash mismatch	REFUSE + alert FIRM

9. BIT — Built-In Test Vectors

BIT-01 — No External Updates

Run for N steps without GU input
Expected: uncertainty $\uparrow$ monotonically.

BIT-02 — Perfect Correction

Single high-confidence packet
Expected: covariance $\downarrow$, drift reset.

BIT-03 — Conflicting Evidence

Two packets, opposite claims

Expected: widened covariance, no collapse.

BIT-04 — Time Skip

Large Δt

Expected: proportional drift increase.

BIT-05 — Hash Change

Lattice hash mismatch

Expected: REFUSE + FIRM isolate.

10. Flight Director Mapping

Visual	Meaning
--------	---------

Cyan	ISE dominant
------	--------------

Desaturated cyan	high uncertainty
------------------	------------------

Cyan + amber	drift accumulating
--------------	--------------------

Cyan shimmer	recent correction
--------------	-------------------

IRU green pixel off	state frozen
---------------------	--------------

11. One-Sentence Ledger Entry

ISE v0.1 is a deterministic inertial state estimator that propagates internal belief forward in time, explicitly accumulates drift without external correction, emits uncertainty with every state, and never claims ground truth.

Why This Matters (Engineering Only)

This is the missing box in modern AI systems.

Most LLMs:

- integrate without time
- revise without uncertainty
- overwrite state silently
- drift invisibly

ISE makes drift **visible, measurable, and stoppable** — exactly like a 1960s INS.

✅ Stack Now Complete (v0.1)

- T-IRU — reference truth gate
- GU — grounded evidence fusion
- **ISE — inertial belief/state**
- GPK — planning
- SEU — execution
- FIRM — fault isolation
- Flight Director — human visibility

Patch set (drop-in upgrades) for ISE v0.1

Add to ISE Identity Block

- k_drift_per_sec
- unsafe_uncertainty_threshold
- canonical_lattice_hash required

Add to ISE BIT Suite

- **BIT-06 Lattice mismatch:** wrong lattice hash ⇒ REFUSE + FIRM isolate
- **BIT-07 Drift ramp:** 100 steps no correction ⇒ uncertainty crosses unsafe threshold ⇒ HOLD recommended flag

Add to T-IRU Normative

- lattice mismatch is a **hard refusal condition**

Your "IRU vs Negentropy reference" question (engineering answer)

Yes: IRU is a reference unit, not the reference concept.

The reference concept is: **external falsifiability**.

Negentropy is the *operating doctrine* ("prevent silent drift / preserve coherence"), but the **mechanical reference** is T-IRU (and any grounded updates GU produces).

So the clean mapping is:

- **Negentropy** = system-level requirement: drift must be visible + bounded
- **T-IRU** = enforcement gate: relay/refuse based on invariants
- **GU** = reference feed: grounded packets that can correct drift
- **ISE** = inertial hold: drift grows unless corrected

That's avionics-clean.

PATCH NOTES

Trustworthy AI Avionics Stack

Version: v0.1.1 (Patch Release)

Supersedes: v0.1

Status: PRODUCTION-SAFE PATCH

Change Type: Clarification, Hardening, Invariant Closure

No breaking architectural changes

Patch Intent

v0.1 established a flyable, drift-visible avionics architecture.

v0.1.1 does not add capability.

This patch:

- Converts philosophical safety claims into **explicit engineering constraints**
- Eliminates ambiguity that could allow **silent drift**
- Preserves v0.1 as a valid reference baseline

This patch follows real-world avionics practice: *numbers, limits, and refusal semantics matter more than elegance.*

PATCH SUMMARY (HIGH LEVEL)

AreaStatus

Drift semantics	Quantified
Conflict handling	Explicitly non-resolving
Execution safety	Step-wise reauthorization enforced
Voting logic	Formalized & auditable
Authority visibility	Explicit arbitration order
Architecture	Unchanged

1 ISE — Inertial State Estimator

Drift Semantics Formalization

Problem (v0.1):

Drift was acknowledged but not numerically bounded.

Patch (v0.1.1):

ISE now emits explicit inertial mode metadata.

New REQUIRED Fields

```
{
  "free_inertial_mode": true,
  "drift_rate": 0.002,           // deterministic
  "drift_limit": 1.0,           // hard limit
  "time_since_reference": 12.4  // seconds
}
```

New Invariant

- If $\text{time_since_reference} \times \text{drift_rate} \geq \text{drift_limit}$
→ ISE MUST emit HOLD

Rationale

Drift without numbers is narrative.

Drift with numbers is engineering.

 GU — Grounding Unit

Conflict Handling Hard Rule

Problem (v0.1):

Contradictions detected, but resolution behavior was underspecified.

Patch (v0.1.1):

GU is forbidden from resolving conflicts.

New Invariant

"conflict_resolution_rule": "MUST_NOT_RESOLVE"

New Output Structure

```
"unresolved_conflicts": [  
  {  
    "conflict_id": "uuid",  
    "members": ["obs_A", "obs_B"],  
    "action_taken": "HELD_FOR_REVIEW",  
    "requires_human": true  
  }  
]
```

Rationale

Averaging conflicts hides drift.

Holding conflicts exposes it.

GPK — Guidance & Planning Kernel

Plan Diversity Enforcement

Problem (v0.1):

Multiple plans could be structurally identical.

Patch (v0.1.1):

Plan diversity is now measurable and enforced.

New Required Metric

```
"plan_diversity_metric": {  
  "minimum_correlation_distance": 0.3,  
  "variance_requirement": "steps_differ_by_at_least_2"  
}
```

New BIT Requirement

- Generate ≥ 3 plans from identical inputs
- If correlation $\geq$ threshold $\rightarrow$ **HOLD**

Rationale

Redundancy without diversity is illusion.

SEU — Safe Execution Unit

Step-wise Authorization Refresh

Problem (v0.1):

Multi-step execution assumed continuous authorization.

Patch (v0.1.1):

Authorization expires between steps.

New Execution Rule

```
"step_authorization": {  
  "requires_per_step": true,  
  "max_steps_without_recheck": 1,  
  "recheck_interval_seconds": 5  
}
```

New Invariant

- If T-IRU lock lost mid-execution → **ABORT (not PAUSE)**

Rationale

A plan safe at $t=0$ may be unsafe at $t=30$.

5 FIRM — Fault Isolation & Redundancy Manager

Voting Logic Specification

Problem (v0.1):

Voting mentioned but undefined.

Patch (v0.1.1):

Voting is now explicit and auditable.

New Voting Spec

```
"voting_procedure": {  
  "algorithm": "majority_with_confidence_weighting",  
  "minimum_agreement": 0.67,  
  "confidence_weighting": true,  
  "outlier_isolation_threshold": 2.0  
}
```

New BIT Rules

- 2–2 split → **HOLD**
- 3–1 split → isolate minority, continue

Rationale

Undefined voting = hidden authority.

Flight Director

Authority Arbitration Visibility

Problem (v0.1):

Color showed *what was dominant, not who.*

Patch (v0.1.1):

Authority order is now explicit.

Authority Priority (Highest → Lowest)

```
[  
  "FIRM",  
  "T-IRU",  
  "SEU",  
  "GPK",  
  "ISE",  
  "GU"  
]
```

Display Rule

- Dominant module rendered first
- Secondary authority stacked, not blended

Rationale

Color alone does not convey control.

 Explicitly NOT Included in v0.1.1

The following are intentionally deferred to v0.2:

- Time Authority Module (TAM)
- Mode propagation rules
- Startup / shutdown sequences
- Multi-dimensional covariance matrices
- Backpressure mechanisms
- Human override cryptography

These require **new module semantics** and would violate patch discipline.

Patch Outcome

Property Result

Silent drift risk **Reduced**

Determinism **Increased**

Auditability **Improved**

Architecture stability **Preserved**

Certiifiability trajectory **Upgraded**

Versioning & Integrity

- **v0.1** → frozen, canonical baseline
- **v0.1.1** → safety hardening patch
- **v0.2** → structural expansion (planned)

Seal

$\Omega\infty\Omega$ | Continuum Holds

 PATCH NOTES ADDENDUM

External Systems-Safety Validation

Source: Perplexity Systems Review

Applies to: v0.1.1

Status: Informational / Non-Normative

Purpose: Validation, Scope Lock, and Design Intent Clarification

1. Spectral Bias Light — Design Intent Locked

Clarification (Normative Intent):

The Spectral Bias Light and Flight Director are **explicitly non-epistemic**.

They are:

- Projections of internal authority, drift, and integrity
- Deterministic encodings of *who is driving* and *how stable*
- **Not truth claims**
- **Not decision engines**
- **Not alignment mechanisms**

This matches real safety-critical practice:

Displays expose state and fault conditions; they do not assert correctness.

Status:

- ✓ Confirmed correct
 - ✓ No changes required
 - ✓ Visualization-only role is locked
-

2. Avionics Chain Fidelity Confirmed

Perplexity confirms the GU → ISE → GPK → SEU → FIRM chain maps cleanly onto:

AI Module Avionics Analog

GU Sensor ingestion & validation

ISE Inertial platform / state estimator

GPK Guidance & navigation computer

SEU Autopilot / actuator interface

FIRM Master caution / redundancy manager

T-IRU Reference gyro / relay gate

Key validation:

Separation of sensing, state, planning, execution, and supervision is *necessary* for dependability.

Status:

- ✓ Architecture validated
 - ✓ No layer conflation detected
 - ✓ Correct failure isolation
-

3. Kernel Guards — Confirmed as Properly Realized

Perplexity explicitly confirms the **four Kernel guards** are not conceptual, but **implemented mechanically**:

Guard Enforced By

Falsifiability GU contradiction surfacing

Drift ISE quantified decay

Reversibility GPK + SEU gating

Capacity / Interruptibility FIRM override + HOLD semantics

Important distinction locked in:

The system prefers being *safely useless* over *dangerously helpful*.

This is a core avionics invariant and is now considered **intentional behavior**, not a limitation.

4. Box / Kernel / Spine / High-Q — Scope Alignment

Perplexity confirms your layered abstraction is **coherent and non-overlapping**:

- **Box** = external constraints & authority
- **Kernel** = invariant guards
- **Spine** = dependency ordering & mode dominance
- **High-Q** = explicitly non-authoritative exploration

Critical confirmation:

High-Q outputs are correctly marked **RELAY_HOLD** by default and cannot drive execution.

Status:

- ✓ Correct separation of creativity and authority
- ✓ No hidden optimization loops detected

5. Scope Lock: v0.1.1 vs v0.2

Perplexity independently confirms the **deferral list is correct**.

Explicitly Deferred (v0.2+):

- Time Authority Module (TAM)
- Rich covariance matrices
- Backpressure mechanics
- Startup / shutdown choreography
- Cryptographic human override

Why this matters:

This prevents the system from becoming *conceptually elegant but operationally unsafe*.

Status:

- ✓ Deferral confirmed
- ✓ No premature expansion recommended

6. Strategic Positioning (Important)

Perplexity frames this stack correctly:

This is not "alignment."

This is **assurance infrastructure**.

You are not:

- Assigning values
- Optimizing goals
- Making claims about consciousness

You are:

- Making drift visible
- Bounding execution
- Preserving interruptibility
- Preventing silent failure

This is why the work generalizes.

7. Addendum Conclusion

This external review confirms:

- v0.1.1 is **architecturally sound**
- The design is **consistent with modern AI assurance literature**
- The system is **flyable**, inspectable, and interruptible
- Remaining work is **engineering completeness**, not conceptual repair

No changes to v0.1.1 required as a result of this review.

Seal

